

COMP 4971C

Optimized News Application to Support the Visually Impaired

COMP 4971C Final Report

By:

KIM Jin Gee

Advised By:

Dr. Desmond Yau-chat Tsoi

Submitted in partial fulfillment of the requirements of

COMP 4971C

in the

Department of Computer Science and Engineering

The Hong Kong University of Science and Technology

2020-2021

Date of Submission:

29 May 2021

Abstract

Over 253 million people in the world are categorized as visually impaired. Although many organizations are working hard towards improving the situation for the visually impaired, there are still many online contents such as news websites that the visually impaired have a hard time using. This project focuses on improving the consumption of news articles for the visually impaired. Text-to-speech implementations are already prevalent, but these solutions can be optimized further to create a more wholesome and efficient experience for the user. By combining text-to-speech, speech-to-text, and sentence extraction solutions together, this project will create a voice-controlled news application that can provide summarization features for a visually impaired user.

Table of Contents:

Abstract	2
1. Introduction	5
1.1 Overview	5
1.2 Objectives	7
Objective 1: Allow the visually impaired to navigate around the application with only their voice.	7
Objective 2: Provide accurate and concise summaries	7
Objective 3: Create an optimized user-interface for the visually impaired	7
1.3 Literature Survey	8
1.3.1 Feedly	8
1.3.2 JAWS	8
1.3.3 COBRA	8
2. Methodology	9
2.1 Design	9
2.1.1 System Design	9
2.1.2 Design choice and rationale	10
2.1.2.1 News article mobile application	10
2.1.2.2 Calling server for text extraction	10
2.2 Implementation	11
2.2.1 Mobile Application	11
2.2.1.1 React Native	11
2.2.1.2 Text-to-Speech	11
2.2.1.3 Speech-to-Text	12
2.2.1.4 News API	13

2.2.1.5 Server	13
2.2.2 News Article Summarization	14
2.2.2.1 Abstractive Summarization	14
2.2.2.2 Extractive Summarization	15
2.2.2.3 Using Extractive Summarization Instead of Abstractive Summarization	15
2.2.4.4 Application Flow and UI	16
2.3 Evaluation	18
Objective 1: Allow the visually impaired to navigate around the application with only their voice.	18
Objective 2: Provide accurate and concise summaries	18
Objective 3: Create an optimized user-interface for the visually impaired	18
2.4 Limitations and Further Study	19
3. Conclusion	19
4. Hardware & Software	20
5. Appendix	21
5.1 Using Flask to Serve our Model	21
6. References	22

1. Introduction

1.1 Overview

According to the Vision Loss Expert Group, in 2015 there were around 253 million people with visual impairment. Although improving technologies are helping reduce the number of people suffering from visual impairment, the increase in the number of the aging population is making the number of visual impaired continue to rise [1]. However, there are many online contents on the internet that are not suitable for consumption to these users. To be more specific, news articles online consist of mostly words, making it difficult for the visually impaired to read on their small mobile phones. Fortunately, there are organizations such as the Hong Kong Blind Union and the Hong Kong Society for the Blind that have support from the Hong Kong government to make sure websites and online contents provide accessibility to the visually impaired also [2]. But most of the solutions provided are simple Text-to-Speech readers that read out the content in a website. One problem with this solution is that it takes much longer for Text-to-Speech readers to read out the content than the human eye.

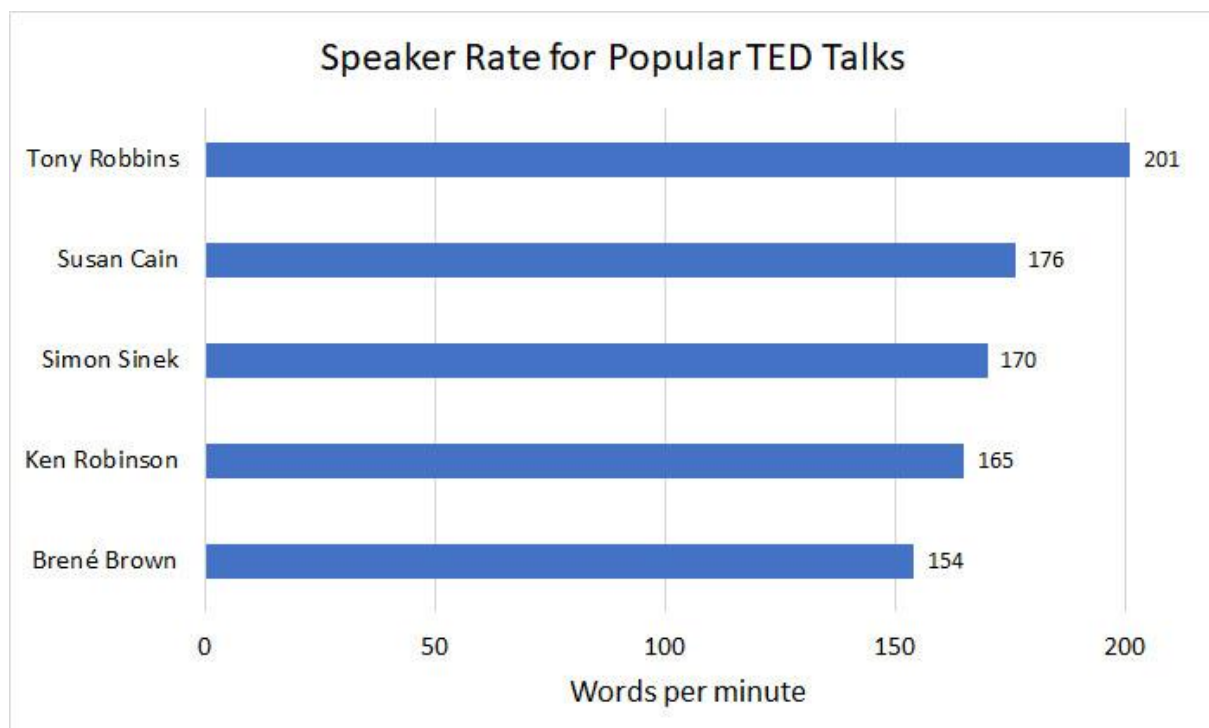


Figure 1 - Speaking Rate [3]

By looking at Figure 1, it can be seen that the speaking rate of a speaker is around 150-200 words per minute. If a visually impaired person wanted to listen to several articles at around 150-200 words per minute, it would take them a considerable amount of time.

Another problem that visually impaired people face is moving around an application or website to find the content that they need. There are several screen readers and refreshable braille displays that allow visually impaired people to find their way around the internet. But since every news website or platform is different, it is tedious for them to find their way around a website and wait for the screen reader program to read each word in the website.

This project will fix these problems by providing a news application that will let visually impaired users to find their way around the application with only their voice and to listen to summaries of news articles so that they can extract the main points of each article and only listen to the full content of articles they are interested in.

1.2 Objectives

The primary objective of this project is to create a mobile application that will provide a better experience for the visually impaired when they are reading the news. The objectives of this project can be separated into the following:

Objective 1: Allow the visually impaired to navigate around the application with only their voice.

The first objective is important because it is difficult for the visually impaired to navigate around a mobile application by pressing on the screen since they cannot locate where the buttons are in the screen. By including a voice assistant that can speak to the user and also receive voice commands, this will allow visually impaired users to navigate around the application.

Objective 2: Provide accurate and concise summaries

For the second objective, it is important to provide users with accurate and concise summaries of a news article. In order to save time for the visually impaired when they are looking for interesting news articles, a concise summary will allow them to quickly know the main point of an article and decide whether they would like to listen to the whole article or skip to the next one.

Objective 3: Create an optimized user-interface for the visually impaired

The last objective is important because although blind people are part of the visually impaired category, most visually impaired people are not totally blind. Therefore, it is important to make sure that the application uses large text and buttons to make it convenient for them to use.

1.3 Literature Survey

After doing exhaustive research, there were no news application specifically for blind people. However, by gaining insight from similar mobile applications, the functionalities of this mobile application could be enhanced.

1.3.1 Feedly

Feedly is one of the most popular news applications out right now. Although this application is not optimized for the visually impaired, it has a summarization feature that makes it easier for users to consume data. The way Feedly's summarization feature works is by highlighting the most important sentences in an article. By doing so, users can skim through the article and only read the highlighted sentences and extract the most important information from the article. This shows the advantage of summarizing news articles in saving time.

1.3.2 JAWS

JAWS is a software screen reading program that provides speech and Braille output [4]. It also provides a feature called Skim Reading that will read the first sentence of each paragraph in order to provide a more efficient way to read the text in a website. The problem with JAWS is that it reads all the content in a website. Therefore, if there are several advertisements in a particular news article or website, the JAWS screen reading program will read all the text in the advertisement as well. On the other hand, the mobile application for this project will only extract the content of a news article and exclude all useless text.

1.3.3 COBRA

COBRA is another screen reader program that helps the visually impaired gain access to text in websites [5]. COBRA's main advantage is that it provides a magnification feature that can magnify content in a website up to 32 times its original size. As many COBRA users enjoy this feature, it comes to show the importance of using large text and that many partially visually impaired people also enjoy using these assistant programs. These ideas can be included into this project. However, the mobile application in this project will provide more functionalities such as supporting voice commands and summarization features.

2. Methodology

2.1 Design

2.1.1 System Design

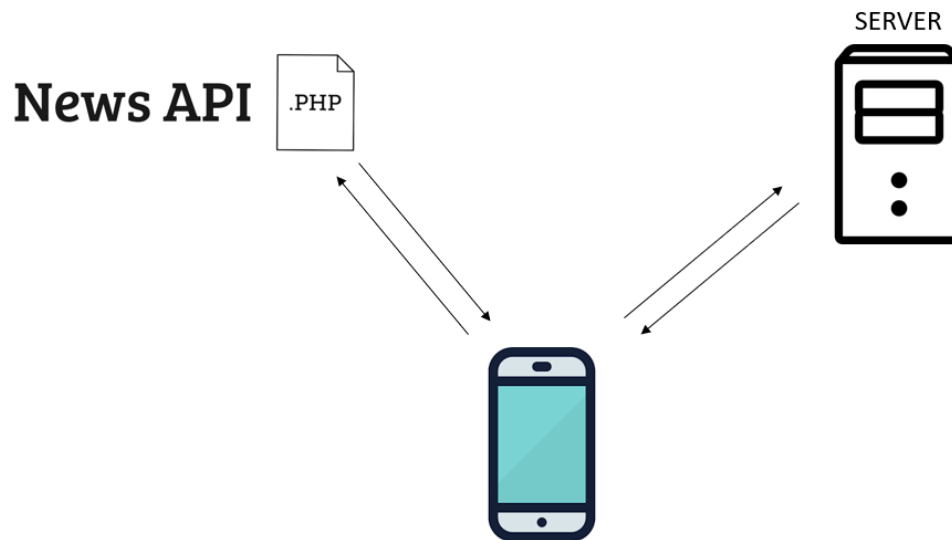


Figure 2 – Design Diagram

The application for this project is made up of three parts: the mobile application, the news API, and the server. The news API is used to load all the articles into the mobile device according to each category. The server holds the components of text extraction and summary extraction. The user interface of the mobile application will provide accessibility to both the news API and the server. Table 1 shows the functions of each component in the diagram.

Component	Function
Mobile Device	Shows the news categories and news articles to the user. Provides voice commands and voice feedback to the user so that all navigations can happen through voice. Calls upon the news API and server.
News API	Uses API provided by https://newsapi.org/ to receive all the latest news.
Server	Extracts text from news articles. Depending on the user's needs, can either return the whole text or a summary of the text.

Table 1 - Table of components included in the system design and their respective functions

2.1.2 Design choice and rationale

2.1.2.1 News article mobile application

Although there are many areas in the internet that can be improved to provide better functionalities for the visually impaired, I wanted to start out with news articles because it seemed like one of the most widely consumed content online and it also contains a massive amount of text that makes it difficult for visually impaired people to consume. Also, by focusing on only news, people can become accustomed to the mobile application and know that if they want to read the news, they can use this application. This will reduce the learning curve people will have to overcome when using a new application.

2.1.2.2 Calling server for text extraction

The reason for using a server for text extraction is to use Python code to do the extraction. The mobile application was constructed using React Native, which uses Node.js to do its coding. However, by using the expansive functionalities of Python, more accurate and efficient text and summary extraction can be accomplished.

2.2 Implementation

The implementation can be first described by exploring the mobile application and then describing the components used within the mobile application.

2.2.1 Mobile Application

2.2.1.1 React Native

The mobile application was developed using React Native. This means this application is available for both IOS and Android devices. Another advantage of using React Native to develop the application is that it provides flexibility when designing the application. Also, since most of the heavy workload is done in the server, there was no need to use a more powerful but more complicated platform like Android Studio. Using a flexible and easy-to-implement platform such as React Native saved time and effort when making the mobile application.

2.2.1.2 Text-to-Speech

Text-to-Speech was integrated into the mobile application by using Google's text-to-speech engine. React Native provides a library that allows linkage to Google's text-to-speech engine [6]. After installing the library, the engine must be initialized. This only takes a fraction of a second.

Set default Language

```
Tts.setDefaultLanguage('en-IE');
```

Set default Voice

Sets default voice, pass one of the voiceId as reported by a call to Tts.voices()

(not available on Android API Level < 21)

```
Tts.setDefaultVoice('com.apple.ttsbundle.Moira-compact');
```

Set default Speech Rate

Sets default speech rate. The rate parameter is a float where where 0.01 is a slowest rate and 0.99 is the fastest rate.

```
Tts.setDefaultRate(0.6);
```

Figure 3 – Default Settings [6]

Figure 3 shows that several default settings can also be changed. By choosing the language, voice, and rate of speech, a better experience can be given to the users.

2.2.1.3 Speech-to-Text

Another functionality of this project is allowing voice commands. This means that if the user speaks to the phone, the speech will be recorded and converted to text. Instead of going through the difficulty of setting up a new speech-to-text machine, Google's Cloud Speech-to-Text API was used. Another React Native library was used to simplify the process of linking to Google's API [7]. Instead of making the Speech-to-Text machine listen for speech all the time, an event listener was implemented so that when the Text-to-Speech machine was finished talking, it would initiate the Speech-to-Text machine.

```
ai() {  
  Tts.getInitStatus().then(() => {  
    Tts.speak('What can I help you with?');  
    Tts.addEventListener('tts-finish', (event) => this.onStartButtonPress());  
  }, (err) => {  
    if (err.code === 'no_engine') {  
      Tts.requestInstallEngine();  
    }  
  });  
}
```

Figure 4 – Event Listener

As can be seen in Figure 4, after the Text-to-Speech machine speaks “What can I help you with?”, an event listener is added that will call the function when the machine is finished speaking.

After the user has finished speaking and the Speech-to-Text machine is done listening, an array is created listing out the possible phrases the user could have said. By using a set of If statements, the correct command can be initiated (Figure 5).

```

{
  if(item === "business")
  {
    Actions.news({topic: 'Business'})
  }
  else if(item === "politics")
  {
    Actions.news({topic: 'Politics'})
  }
  else if(item === "sports")
  {
    Actions.news({topic: 'Sports'})
  }
  else if(item === "tech")
  {
    Actions.news({topic: 'Tech'})
  }
  else if(item === "health")
  {
    Actions.news({topic: 'Health'})
  }
}
})

```

Figure 5 – Command Initiation

2.2.1.4 News API

News API is a platform that provides an API to get news articles. By using REST API in React Native, the articles can be extracted easily. First, an API key must be given. This can be found in the News API website. Then by stating what category news that needs to be targeted, the correct news array can be extracted (Figure 6).

```

async componentDidMount() {
  await fetch('https://newsapi.org/v2/everything?domains=cnn.com&q='+ this.topic + '&pageSize=10&apiKey=29ed1409583c4ceb80b91ebd911c015'
    method: 'GET'
  })
  .then((response) => response.json())
  .then((json) => {
    var x = json['articles'];
    x.forEach(element => {
      this.setState({ title: [...this.state.title, element] })
    });
  });
}
var ind = 1;
this.state.title.forEach(element => {
  this.speak(element.title, ind)
  ind = ind + 1;
});

```

Figure 6 – API to get News

2.2.1.5 Server

The server was hosted by using a platform called Flask. Flask will package Python code and upload it to the server, making it possible to use Python libraries. Then, the domain of the server can be called on using REST API in the mobile application. Please refer to *Appendix 6.1* for further details.

2.2.2 News Article Summarization

Two approaches were experimented in order to find the best method of summarizing news article text. The first approach is a machine learning approach using data preprocessing and LSTM algorithms. The second approach is using PageRank to rank the most important sentences in the news article. These two approaches were implemented and compared to see which would be most effective for news article summarization. For both approaches, a Python library called “Newspaper3k” was used to extract the main content from a news article [9]. This will make sure all unnecessary text content from the news articles is removed.

2.2.2.1 Abstractive Summarization

The machine learning approach can be described as abstractive summarization because instead of using already existing sentences, it is attempting to construct a new summary sentence. To begin implementing the machine learning approach, a large dataset of text phrases with its corresponding summaries was needed. This could be found on Kaggle and consisted of new articles text and the summary associated with it. Then this went through the normal data preprocessing techniques such as removing contractions, removing stopwords, and performing tokenization. The machine learning model that was used included an embedding layer (which is used for many text-associated machine learning models), four LSTM layers, and an attention layer as can be seen in Figure 7.

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(None, 80)]	0	
embedding (Embedding)	(None, 80, 500)	25785500	input_1[0][0]
lstm (LSTM)	[(None, 80, 500), (N 2002000		embedding[0][0]
input_2 (InputLayer)	[(None, None)]	0	
lstm_1 (LSTM)	[(None, 80, 500), (N 2002000		lstm[0][0]
embedding_1 (Embedding)	(None, None, 500)	7048000	input_2[0][0]
lstm_2 (LSTM)	[(None, 80, 500), (N 2002000		lstm_1[0][0]
lstm_3 (LSTM)	[(None, None, 500), 2002000		embedding_1[0][0] lstm_2[0][1] lstm_2[0][2]
attention_layer (AttentionLayer	((None, None, 500), 500500		lstm_2[0][0] lstm_3[0][0]

Figure 7 – Summarization Model

However, the results were dissatisfactory as the validation loss was extremely high and results on the test set was poor. This may be due to the sensitivity of working with a text-based dataset and the difficulty of constructing a brand-new sentence.

2.2.2.2 Extractive Summarization

Extractive summarization is the process of extracting the most important sentences from the news article. In this project, the process of ranking the most important sentences was done through PageRank. First, the common data preprocessing methods were used such as separating the news article paragraphs to sentences, removing punctuations, removing stopwords, and converting the sentences to vectors. Afterwards a similarity matrix was created and the sentence vectors mapped to a network graph. Then, cosine similarity was used to determine how similar the sentences are to each other, which will show which sentences are most relevant in the news article.

2.2.2.3 Using Extractive Summarization Instead of Abstractive Summarization

Although the machine learning model could have been developed to produce better results, this method was quickly dropped as it would only waste time and effort. This is because in most news articles, there exists sentences that already state the main points of the news article. By using extractive summarization, these sentences can be pinpointed and extracted which will make the summarization process much more efficient and simpler.

2.2.4.4 Application Flow and UI



Opening Screen

The opening screen shows the news categories that are available in this mobile application. As soon as this screen is shown, the voice assistant will ask “Which news category would you like to go to?” Then, the user can verbally say the category. For example, if the user says “Business”, the mobile application will navigate to the “Business Category”. Anytime the user would like to activate the voice assistant, the floating button on the bottom right corner can be pressed.

Figure 8 - Opening Screen

News Page

The news page shows the top ten news articles for each category. Ten articles were specifically chosen because the user would probably wish to look at the most important news for that day. However, if it becomes too long, it will take a long time for the user to hear all the headlines for each article. As soon as this page is opened, the voice assistant will start reading the article titles. Anytime the user would like to open an article, the floating button on the bottom right corner can be pressed. The voice assistant will say “What can I help you with?”. The user can simply say “Open article _” (fill the blank with article number) to open an article.

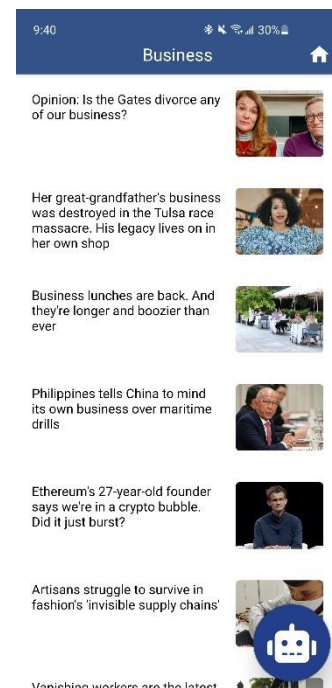


Figure 9 - News Page



Figure 10 - Article Page

Article Page

This page shows the content of the news article that was opened. As soon as this page is opened, the voice assistant will say “Would you like to hear everything or just the summary?”. The user can reply with “everything” or “summary”. “Everything” will prompt the voice assistant to read the whole article. “Summary” will prompt the assistant to call on the summary to get summary of the article and read it out to the user.

2.3 Evaluation

Objective 1: Allow the visually impaired to navigate around the application with only their voice.

By integrating Text-to-Speech features into the application, the user can successfully navigate around the application by only using their voice. Furthermore, the voice assistant will ask the users specific questions so that the visually impaired users will know how to reply.

Objective 2: Provide accurate and concise summaries

Even though the articles were several paragraphs long, the content was able to be summarized to a couple of sentences or less. Also, even though the machine learning method was not accurate, the extractive summarization method was able to pick out sentences in the text that was relevant to the main topic of the article.

Objective 3: Create an optimized user-interface for the visually impaired

Even though most of the navigation is done through voice, the mobile application still contains big buttons that can be easily pressed. For example, in the opening page, the categories are written in large text with even larger boxes around them so that it can be seen easily. This was tested by showing friends and family members who have bad eyesight and asking them if the text was large enough. They were able to confirm that it was sufficient.

2.4 Limitations and Further Study

Voice Assistant

Currently, the voice assistant needs to hear the exact commands from a user to perform a task. However, this can be improved to allow the voice assistant to understand a command even though the user does not say the exact phrase. This could be fixed by collecting a dataset of possible commands a user could say and training a machine learning model to interpret what a user is trying to convey.

Latency of Server

It takes around 5-10 seconds for the server to return a summary to the user. This would be unacceptable for commercial use. The reason the server takes a long time is because of network speed and mapping the text to a network graph and calculating the cosine similarity. If this algorithm could be optimized or by using a machine learning model, the time could be reduced significantly.

3. Conclusion

By using a mixture of different techniques, I was able to successfully create a voice-powered mobile application that could deliver news to the visually impaired. They can open the app, navigate to an article using only their voice, and choose to listen to a summary or the whole article. The reason this topic was chosen was because I wanted to create an application that could somehow help society. And because the number of blind people is continuing to increase, I was hoping this mobile application could help the visually impaired. Although there are still many limitations with the application, I hope this can be the beginning of my goal to improve the society I am living in.

4. Hardware & Software

Hardware				
Product	Specification	Usage	price (HKD)	Quantity
Mobile device	Specification may defer	Test application	700~10,000HKD	1
Laptop	SAMSUNG Laptop	Train machine learning	Free	1

Software			
Product	Version	Usage	Price
React Native	0.63	Mobile Application Development	Free
Flask	1.1	Summarization Deployment Server	Free
News API	N/A	API to get news	Free for Developers

5. Appendix

5.1 Using Flask to Serve our Model

The steps to get started with Flask is documented clearly in the Flask website [64]. This section of the appendix will describe how to set up Flask in relation to our project.

Before using Flask, we first need to set up a virtual python environment and install Flask in this environment. This can be done by following the instructions written in the Flask website [64]. By setting up a virtual environment, we will be able to set the Python version and library versions to exactly what we need.

Our team has created a separate Python file that can receive inputs through HTTP requests, load the model, and produce an output. This file contains the necessary TensorFlow and Keras libraries. Therefore, we also need to install these libraries into our virtual environment. In order to install all these libraries, make sure to have a version of Python that is 3.6 or greater.

Now we can include the following code to set up the Flask server:

```
app = Flask(__name__)
CORS(app)

@app.route("/", methods = ['POST', 'GET'])
def predict():
```

The first line is simply creating a Flask instance with the name of our web application. The second line allows requests to be made on the server without interference from security protocols. Now, the third line of code is telling Flask what URL should be used for the server. For example, since our web application is currently routed to "/", our URL will be <http://domainname/>. If our web application was routed to "/example", the URL would be <http://domainname/example>. Finally, we can include our code to run our model in the predict function, and call this function when the URL is called upon.

Now all we need to do is run our Python file, and the server will begin to start running.

```
(venv) C:\Users\jinge\Documents\flask>python model.py
2021-04-13 23:09:24.614923: W tensorflow/stream_executor/platform/default/dso_loader.cc:60] Could
not load dynamic library 'cudart64_110.dll'; dlderror: cudart64_110.dll not found
2021-04-13 23:09:24.616303: I tensorflow/stream_executor/cuda/cudart_stub.cc:29] Ignore above cud
art dlderror if you do not have a GPU set up on your machine.
* Serving Flask app "model" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: on
* Restarting with stat
2021-04-13 23:09:31.916098: W tensorflow/stream_executor/platform/default/dso_loader.cc:60] Could
not load dynamic library 'cudart64_110.dll'; dlderror: cudart64_110.dll not found
2021-04-13 23:09:31.916528: I tensorflow/stream_executor/cuda/cudart_stub.cc:29] Ignore above cud
art dlderror if you do not have a GPU set up on your machine.
* Debugger is active!
* Debugger PIN: 781-759-604
* Running on http://0.0.0.0:5000/ (Press CTRL+C to quit)
```

6. References

- [1] P. Ackland, S. Resnikoff, and R. Bourne, "World blindness and visual impairment: despite many successes, the problem is growing," *Community eye health*, 2017. [Online]. Available: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5820628/>. [Accessed: 29-May-2021].
- [2] G. H. K. (www.gov.hk), "GovHK: Organisations & Websites for Persons with Disabilities," *GovHK 香港政府一站通*, 02-Sep-2020. [Online]. Available: <https://www.gov.hk/en/residents/communication/visuallyimp/disabilities.htm>. [Accessed: 29-May-2021].
- [3] D. Barnard, "Average Speaking Rate and Words per Minute," *VirtualSpeech*, 20-Jan-2018. [Online]. Available: <https://virtualspeech.com/blog/average-speaking-rate-words-per-minute>. [Accessed: 29-May-2021].
- [4] "JAWS[®]," *Freedom Scientific*. [Online]. Available: <https://www.freedomscientific.com/Products/software/JAWS/>. [Accessed: 29-May-2021].
- [5] "Combined Speech, Braille and Magnification: Experience the Elegance and Functionality of a German-Engineered Screenreader," *Cobra: Screen Reader with Braille, Speech and Magnification*. [Online]. Available: <http://www.bayareadigital.us/products/baum/cobra.html>. [Accessed: 29-May-2021].
- [6] "react-native-tts," *npm*. [Online]. Available: <https://www.npmjs.com/package/react-native-tts>. [Accessed: 29-May-2021].
- [7] React-Native-Voice, "react-native-voice/voice," *GitHub*. [Online]. Available: <https://github.com/react-native-voice/voice>. [Accessed: 29-May-2021].
- [8] K. Vonteru, "NEWS SUMMARY," *Kaggle*, 13-Nov-2019. [Online]. Available: <https://www.kaggle.com/sunnysai12345/news-summary>. [Accessed: 29-May-2021].
- [9] "newspaper3k," *PyPI*. [Online]. Available: <https://pypi.org/project/newspaper3k/>. [Accessed: 29-May-2021].